

26F Stat TA Session W1



Welcome to the TA session of ECON2022!

0. What are we going to do in the TA session?

- Learn how to use `R` for statistical analysis!
 - basic syntax
 - data manipulation
 - data visualization
- In-class practice of `R`
- Quizzes (check the syllabus for more details)



BTW: If things start looking a little *too* bright, hit `Ctrl+Shift+L` for dark mode

▼ Q. Why should we learn coding when LLMs are already better than us?

Ans.

1. Basic knowledge of how to code saves us time costs (or tokens) to co-work with AI
2. Also save you the effort to proofread and check where your code went wrong!
3. Because you have to finish the problem sets (which includes `R` practices)

1. Where can you find class materials/ TAs?

- Always check the **course websites** and announcements first!
 - NTU Cool (mainly for the lectures)
 - [TA session website](#) (click the link and explore the website!)
- TA contact information
 - Leading TA, Edward (陳品勳): R15323005@ntu.edu.tw
 - Office hours: By appointment
 - Please email with a subject prefix `[Statistics TA]` to nudge me to reply sooner
 - Feel free to discuss with TAs if you have any questions about the lecture (statistics) or `R` in office hours (or class break)
 - Other two TAs

- 陳昶安: R15323001@ntu.edu.tw
- 王仁牧: R15323018@ntu.edu.tw

2. First class: Install R & Know R

What do we need?

- R is a free and open-source programming language for **statistical computing** and **data analysis**, specifically
- Besides, we need RStudio to edit our own codes

Download and Install

- Step 1: [Click](#) to download R
- Step 2: [Click](#) to download RStudio

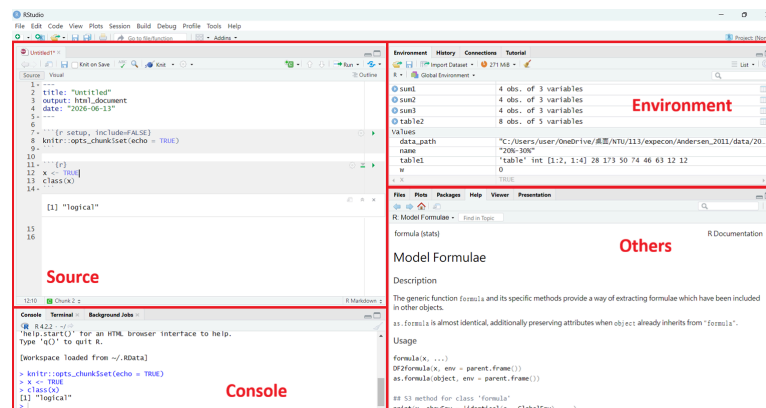
Personal Recommendation

- I highly recommend you to check CS50R on YouTube, which gives a wonderful introduction to R
- But you are still expected to attend TA session and complete the in-class exercise...

https://youtube.com/playlist?list=PLhQjrBD2T382yfNp_-xzX244d-O9W6YmD&si=zz2MBJnQTd8CvZxk

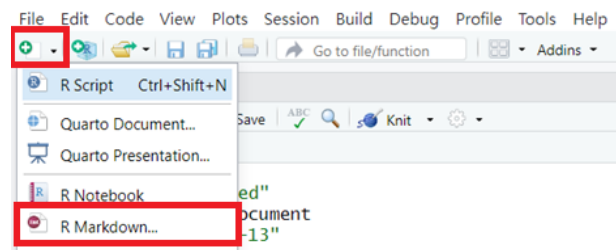
RStudio Interface

- Once you create a file, the interface should look like the figure below

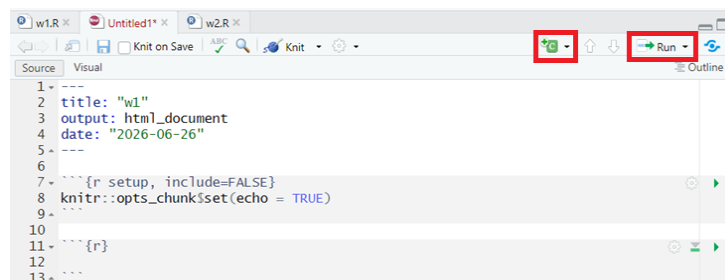


- There are 4 blocks
 - Source: this is where you write and edit your code
 - Environment: check your data and objects here
 - Console: see your output here
 - Others (Files/ Plots/ Packages/ Help etc.): help is the most important one

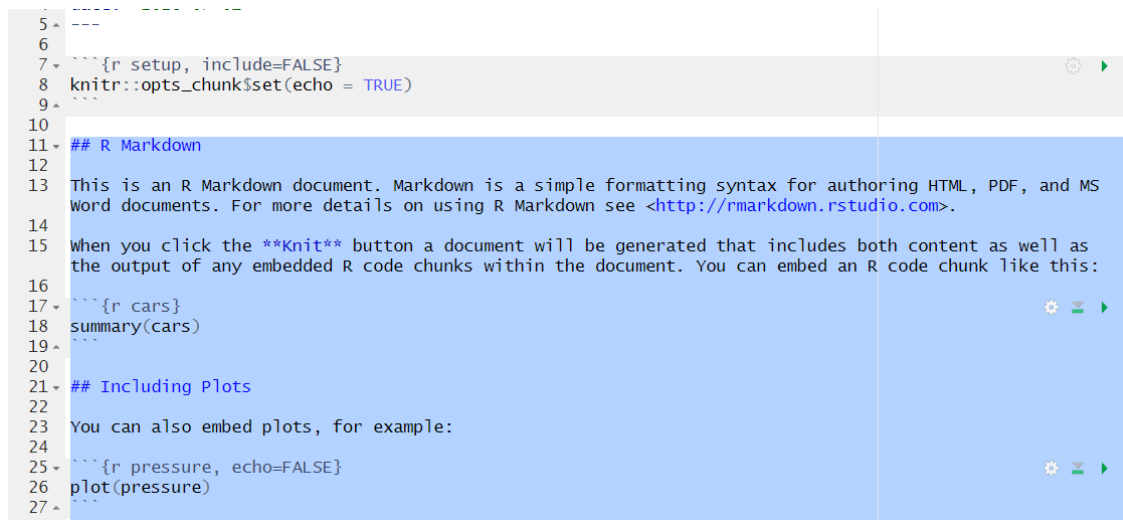
Create a new file



- Step 1: Click the icon with a  on the right above side
- Step 2: Select R Markdown ()



- Step 3: you may delete the unnecessary parts (coding examples for )



- Step 4: add new  code chunk to type your code
- Step 5: run your code by clicking 

3. Basics

Assign values

In **R**, we use **<-** (assignment operator) to assign value to variable

```
# example  
a <- 3
```

A value assigned to a variable can be a number, a character, a dataframe, or the result of an expression

```
# example  
b <- a*a  
# b = 9  
  
x <- "hello"  
# x = "hello"
```

▼ Q. Can we use **=** to assign values? What's the difference?

Ans. It works sometimes

```
# the following are the same  
x <- 5  
y = 5  
  
# but the following are not the same  
mean(x <- 1:10)  
x # [1] 1 2 3 4 5 6 7 8 9 10  
mean(x = 1:10)  
x # Error: object 'x' not found
```

Check the post below if you want to learn more, but the key idea is to



use **<- instead of **=** to avoid ambiguity!**

What are the differences between "=" and "<-" assignment operators?

What are the differences between the assignment operators = and <- in R?

I know that operators are slightly different, as this example shows

 <https://stackoverflow.com/questions/1741820/what-are-the-differences-between-and-assignment-operators>

4. Data Types/ Data Structures

Data Types

- There are several data types in `R`
 - Numeric: numbers with decimal points
 - Character: text
 - Logical: `TRUE` or `FALSE`
- We can use the `class()` function to check the data type of a variable

```
# example
x <- 5
class(x) # Output: "numeric"
y <- "Hello World!"
class(y) # Output: "character"
z <- TRUE
class(z) # Output: "logical"
```

Data Structures

- We can store data in the following ways
 - Vector: one-dimensional sequence of values of the **same** data type

```
# example
id <- 1:4 # 1 2 3 4
class(id)
# Output: "integer"

score <- c(30, 86, 50, 95)
class(score) # Output: "numeric"

name <- c("Clark", "Lois", "Bruce", "Luthor")
class(name) # Output: "character"
```

- List: one-dimensional sequence of values with possibly **different** data types

```
# example
my_list <- list("Hello World!", 1, TRUE, score)
print(my_list)

# Output
[[1]]           # the 1st element in the list
[1] "Hello World!"

[[2]]
[1] 1
```

```
[[3]]
[1] TRUE

[[4]]           # a list is one-dimensional, but it can contain ve
ctors!
[1] 20, 86, 50, 95
```

? Why can a list store vectors, or even dataframes?

- Matrix: two-dimensional/ same data type

```
# example
id <- 1:4
score <- c(30, 86, 50, 95)
id_score <- matrix(c(id, score), nrow = 4)

# Output
print(id_score)
      [,1] [,2]
[1,]    1   30
[2,]    2   86
[3,]    3   50
[4,]    4   95
```

- Dataframe: two-dimensional/ possibly different data type

```
# example
name <- c("Clark", "Lois", "Bruce", "Luthor")
score <- c(30, 86, 50, 95)
df <- data.frame(stu_name = name,
                 stu_score = score)

# Output
  stu_name stu_score
1   Clark         30
2    Lois         86
3   Bruce         50
4  Luthor         95
```

- We categorize them in the following table

	same data structure	possibly different data structure
1-dimensional	vector	list
2-dimensional	matrix	dataframe
more dimensions	array	

- There are some other special data structures
 - Array: more dimensions/ same data structure
 - Factor: ordered vector
 - Tibble: refined version of dataframe in `tidyverse` package

5. Download packages/ Seeking for help

Packages

- In later classes, we will be introduced to different packages in `R`, always remember to
 - Step 1: Use `install.packages("package_name")` to install packages
 - Step 2: Load a package into your `R` session with `library()`

```
# example
install.packages("tidyverse") # add ""

library(tidyverse) # no "" needed
```

help

- Sometimes you may forget the syntax of a specific function, use `help()` or `?` to check the document (it will show up in the "others" block)

```
# example
help(mean)
?mean
```

Another way to seek help

- Take advantage of LLMs (they are good enough to handle basic questions!)
- Feel free to ask TAs!

Before you leave...

Try to finish the exercises on your own!

W1 Exercises